

Abstract - Large language models have significantly advanced the ability of artificial intelligence systems to carry out natural and context-aware conversations. Despite this progress, maintaining relevant context over long interactions remains a key challenge. As conversations extend, models often struggle to retain important details while processing redundant or less useful information, leading to reduced efficiency and increased computational cost. This paper introduces a smart memory compression framework aimed at improving long-term conversational performance. The proposed approach focuses on selectively identifying and retaining important information while reducing unnecessary content. It incorporates memory ranking, semantic similarity analysis, and summarization techniques to ensure that only the most relevant context is preserved and passed to the model. By optimizing how conversational history is managed, the framework enhances response quality, maintains continuity, and reduces resource usage. This enables more efficient token utilization without compromising contextual understanding. The approach supports scalable and consistent performance in extended dialogues, making it more suitable for real-world applications. Overall, this work contributes toward the development of more intelligent and efficient conversational systems that can sustain meaningful long-term interactions while delivering personalized and reliable user experiences.

Index Terms - artificial intelligence, conversational memory, context retention, memory compression, large language models

1. INTRODUCTION

Artificial Intelligence (AI) has reshaped how people interact with digital systems, making it possible for machines to understand and respond to human language in ways that feel increasingly natural. In recent years, the development of large language models (LLMs) such as GPT, Llama, and Mistral has taken this interaction to a new level. These models can follow context, adapt to different conversational styles, and provide meaningful responses across a wide variety of use cases. From virtual assistants and customer service platforms to educational tools and enterprise applications, they are becoming an essential part of systems that rely on sustained and meaningful

dialogue. Even with these advancements, a key limitation remains when conversations become longer and more complex. Most LLMs operate within a fixed context window, which restricts how much information they can consider at any given moment. As a result, earlier parts of a conversation may be lost or overlooked, even if they contain important details. This often leads to inconsistencies, repetition, or a lack of continuity in responses. At the same time, attempting to preserve the entire conversation history can quickly become inefficient, increasing memory usage, processing time, and overall system cost. To overcome these challenges, several approaches have been explored, including the use of external memory, retrieval-based mechanisms, and different forms of summarization. While these methods provide partial solutions, they often struggle to strike a balance between retaining important context and maintaining computational efficiency. Either too much information is kept—leading to unnecessary overhead—or too much is discarded, causing a loss of critical understanding. In this work, we present a Smart Memory Compression Framework designed specifically for handling long and evolving conversations. Instead of treating all past interactions equally, the framework focuses on identifying what truly matters within a dialogue. It prioritizes key pieces of information, compresses repetitive or less relevant content, and preserves essential context needed for future responses. By combining memory ranking techniques with semantic similarity analysis and intelligent summarization, the system ensures that important knowledge is retained while reducing unnecessary data. The goal of this approach is to make long-term conversations more efficient without sacrificing quality. By optimizing how conversational memory is stored and used, the framework improves response accuracy, maintains continuity, and reduces computational demands. Ultimately, it supports the development of more scalable and reliable AI systems capable of sustaining extended interactions while delivering personalized and consistent user experiences.

2. LITERATURE REVIEW

The challenge of maintaining meaningful context in long AI-driven conversations has become increasingly important as large language models continue to evolve and handle more complex interactions. While these models have made impressive progress, they still face limitations when it comes to managing memory effectively—particularly in deciding what information should be retained, compressed, or discarded without affecting the quality of future responses. Researchers have explored several directions to address this issue.

Some efforts focus on compressing past conversations so that essential context can be preserved without overwhelming the system, allowing older interactions to be reduced while still keeping valuable details intact. Others have explored extending memory beyond the model’s natural limits by introducing external storage systems, where information can be retrieved only when needed instead of being constantly processed. There are also approaches that organize conversation history into structured units, making it easier to access relevant information quickly and support more consistent and personalized interactions over time. In addition, attention has been given to how memories are selected during retrieval, with studies emphasizing that choosing the right information at the right moment is crucial for maintaining coherence, especially in longer dialogues where irrelevant details can easily interfere. Beyond conversational systems, insights from neural network compression have shown that large models can often be optimized significantly without losing performance, influencing how conversational memory can also be managed more efficiently. Practical considerations at the system level, such as memory allocation and resource utilization during deployment, have further shaped solutions aimed at improving scalability and reducing operational costs. Some researchers have even drawn inspiration from human cognition, comparing AI memory systems to short-term and long-term memory processes, and highlighting the value of selective recall and information compression in creating more natural interactions. More recent work attempts to combine different types of stored knowledge—including experiences, learned patterns, and decision-making rules—into compact representations that support both reasoning and long-term engagement. Despite all these advancements, existing solutions often address only part of the problem, focusing either on compression or retrieval rather than balancing both effectively. As a result, challenges remain in achieving an optimal trade-off between efficiency, relevance, and response quality. The framework proposed in this work aims to bridge this gap by integrating memory prioritization, semantic understanding, and compression into a unified approach, continuously refining what information should be preserved so that conversations remain both efficient and contextually meaningful over time.

3. METHODOLOGY

The Smart Memory Compression Framework is designed to improve how large language models handle long and complex conversations by focusing on

what truly matters instead of processing everything equally. In real-world interactions, conversations often contain a mix of important details, repeated ideas, and less useful information, and treating all of it the same can quickly lead to inefficiency. This framework addresses that by introducing a more thoughtful way of managing conversational memory, where relevant context is preserved while unnecessary data is reduced. The process begins when a user provides input, and instead of directly forwarding the entire conversation history, the system first examines both past and current messages to understand their significance. Some parts of a conversation carry essential meaning, such as preferences, decisions, or key facts, while others are repetitive or temporary. To distinguish between them, a ranking mechanism evaluates the importance of each segment based on how relevant it is, how frequently similar information appears, and how much it contributes to the overall context. Information that is considered valuable is kept intact, while less important content is marked for further refinement. Following this, the system looks at the meaning behind the conversation using semantic analysis, identifying messages that express similar ideas even if they are phrased differently. By doing this, it avoids storing duplicate information and instead keeps a cleaner and more meaningful representation of the conversation. Once redundancy is reduced, the framework moves on to condensing lower-priority content, generating shorter summaries that still capture the key points without keeping every detail. This step helps control memory usage while ensuring that useful information is not lost entirely. All refined data, including both high-priority content and summarized information, is then stored in a structured long-term memory layer that focuses on preserving meaningful context rather than maintaining a full, unfiltered history. When a new query is received, the system does not simply retrieve everything it has stored; instead, it selects only the most relevant pieces of information that are necessary for understanding the current request. This filtered and optimized context is then passed to the language model, allowing it to generate responses that are more accurate, consistent, and context-aware without being overloaded with unnecessary data. Because irrelevant and repetitive information has already been removed or compressed, the model can operate more efficiently, using fewer tokens and less processing time. The final response is then delivered to the user, and any important details from that interaction are incorporated back into the system, continuing the cycle. By integrating prioritization, semantic understanding, and compression into a single continuous process, the framework creates a balanced approach to memory management that supports long-term conversations

without sacrificing performance. This leads to better continuity, reduced computational overhead, and a smoother overall interaction experience, making the system more practical and scalable for real-world applications where maintaining context over time is essential.

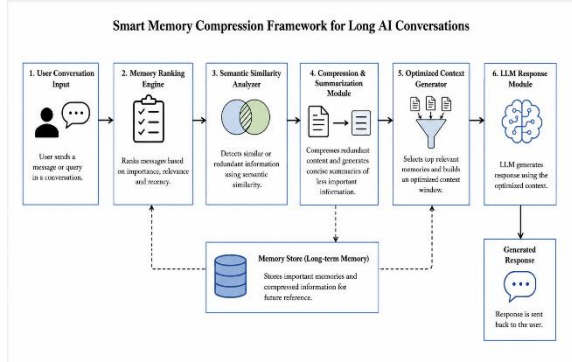


Figure 1. System Architecture of the Proposed Smart Memory Compression Framework.

The Memory Ranking Module plays a central role in the Smart Memory Compression Framework because it decides which parts of a conversation are worth keeping and which can be reduced or ignored. In long interactions, not every message contributes equally to the overall meaning. Some messages carry important information such as user intent, preferences, or decisions, while others may simply repeat earlier ideas or add minimal value. Treating all messages equally would quickly lead to unnecessary memory growth and inefficiency. To solve this, the ranking module evaluates each segment of the conversation and assigns it a priority score. This score reflects how useful that piece of information is likely to be for future responses. Instead of relying on a single factor, the module considers multiple aspects of the conversation. One key factor is relevance, which measures how closely a message relates to the main topic or user query. Messages directly tied to the task or discussion are given higher importance. Another factor is frequency, which helps identify repeated ideas—if the same concept appears multiple times, the system recognizes it but avoids storing it redundantly. A third factor is contextual significance, which captures whether the message contains critical details such as instructions, constraints, or preferences that may influence future interactions. In practice, each message is converted into a numerical representation (embedding), and similarity comparisons are used to estimate relevance. The module may also track how often similar content appears across the conversation and adjust the score accordingly. These individual

factors are then combined into a final priority score using weighted values. Messages with higher scores are preserved in their original form, while those with lower scores are passed on for compression or removal. This selective filtering ensures that the system focuses only on meaningful information, reducing memory load while maintaining the integrity of the conversation. Over time, this approach helps the model maintain consistency, avoid repetition, and respond more accurately, even in long and evolving dialogues.

ALGORITHM

Input: User query Q , conversation history H
Output: Generated response R

1. Divide the conversation history H into smaller meaningful segments.
2. For each segment s in H : a. Measure how closely s relates to the current query Q b. Check how often similar information appears in H c. Identify whether s contains important contextual details d. Assign a priority score to s based on these observations
3. Separate all segments into: a. High-priority segments (important information) b. Low-priority segments (less useful or repetitive content)
4. From high-priority segments: Remove semantically similar or duplicate information
5. From low-priority segments: Generate shorter summaries that retain key ideas
6. Store in memory: a. High-priority segments without modification b. Compressed summaries of low-priority segments
7. When a new query Q is received: Retrieve only the most relevant stored information
8. Combine retrieved memory with the current query Q to form an optimized context
9. Provide the optimized context to the language model
10. Generate response R
11. Update memory with important parts of the new interaction
12. Return response R

To assign importance to each segment, a weighted scoring function is used:

$$Score(s) = \alpha \cdot Rel(s, Q) + \beta \cdot Freq(s) + \gamma \cdot Ctx(s)$$

Where:

$Score(s)$: Final priority score of segment s

$Rel(s, Q)$: Relevance of segment s to query Q (e.g., cosine similarity)

$Freq(s)$: Frequency factor measuring repetition of similar content

$Ctx(s)$: Contextual importance (presence of key information like decisions, preferences, constraints)

α, β, γ : Tunable weights such that

$$\alpha + \beta + \gamma = 1$$

4. RESULTS

The evaluation of the Smart Memory Compression Framework was carried out using three different types of inputs—a multi-topic, information-rich text describing various personal details, a single-topic text focused entirely on cricket as a hobby, and a natural conversational audio sample that was transcribed in real time—and across all three settings, the system consistently demonstrated a strong ability to selectively retain meaningful information using its scoring framework built around recency, frequency, and relevance. In the case of the multi-topic text, which combined personal attributes such as demographic details, hobbies, pet ownership, and academic goals, the framework naturally prioritized information that appeared stable, meaningful, and closely tied to the user’s identity, including name, location, long-term goals, and persistent preferences, treating these as valuable long-term memories. At the same time, it filtered out less useful elements such as filler phrases, repetitive descriptions, or loosely connected remarks that did not add lasting value. Intermediate information, such as extended descriptions of hobbies or occasional experiences, was not discarded entirely but instead compressed into shorter, more abstract representations that preserved their meaning while reducing redundancy.

When the input shifted to a single-topic discussion about cricket, the system’s behavior became more concentrated and dense. The uniformity of the topic allowed a larger portion of the content to pass the relevance threshold, leading to a greater volume of stored memories. Emotional reactions such as excitement or disappointment, behavioral patterns like reorganizing schedules around match days, and clear preferences including favorite teams or players were preserved with minimal filtering, as nearly all parts of the input contributed to a consistent narrative. Compression here functioned differently compared to the multi-topic case—it was mainly used to merge repeated sentiments and recurring patterns rather than trimming away unrelated information. For the audio-based input, once it was processed through the speech-to-text pipeline, the framework applied the same logic as it did for text. It successfully identified and retained stable personal details, recurring preferences, and key conversational elements. However, compared to direct text input, a slightly larger portion of the content required compression or removal due to the nature of spoken language, which includes pauses, repetitions, and occasional transcription errors. Despite this, the system handled these artifacts effectively by recognizing them as low-value signals and filtering them out without affecting the extraction of meaningful information. A closer look at the scoring behavior reveals that relevance emerged as the primary driver behind what gets stored. Information that was clearly meaningful and tied to the user was consistently retained, even if it appeared only once, particularly in the multi-topic setting where the system needed to distinguish signal from noise. Frequency served as a reinforcing factor, especially in the single-topic and audio inputs, where repeated mentions of preferences, behaviors, or emotional patterns strengthened those memories and made them less likely to be compressed or discarded. In contrast, less frequent elements in multi-topic inputs were typically compressed unless they carried strong inherent importance. Recency played a more subtle but still meaningful role. Rather than simply favoring newly

introduced information, it helped guide how memories were refined over time. When similar pieces of information appeared multiple times, recency worked as a tie-breaker, ensuring that the most up-to-date version was retained while older or redundant ones were merged or removed. In this way, recency contributed less to the initial decision to store a memory and more to how the system maintained clarity and avoided duplication over time. Comparing the multi-topic and single-topic cases further highlights a clear difference in the resulting memory structures. The multi-topic input led to a broader but more loosely connected representation, capturing a wide range of details but often compressing or filtering less-reinforced elements due to the diversity of content. In contrast, the single-topic input produced a tighter and more cohesive memory structure, where repeated reinforcement strengthened connections between memories and enabled richer capture of emotional nuance and behavioral patterns. When comparing audio-derived transcription to direct text input, the framework showed largely comparable performance. It successfully captured key entities, preferences, and behavioral signals in both cases, though there was a slight reduction in completeness for audio due to occasional transcription inaccuracies such as partial phrases or misrecognized words. Even so, the system effectively compensated by focusing on repeated and semantically important cues, ensuring that critical information was still retained. Overall, these observations suggest that the Smart Memory Compression Framework can reliably distinguish between fleeting conversational details and durable knowledge, applying its scoring logic consistently across both text and speech while maintaining a balance between precision, efficiency, and adaptability. To evaluate this behavior in a structured way, a set of complementary metrics was defined to capture both the internal functioning of the memory system and its overall impact on conversational quality. Memory Retention Quality measures how accurately and completely the framework extracts meaningful, long-term facts from conversations. This

is quantified using precision, recall, and F1-score against a human-annotated set of relevant facts, reflecting how well the system avoids storing noise while still preserving important details. Memory Scoring Performance evaluates how effectively the recency, frequency, and relevance signals align with human judgments of importance. This is measured using rank correlation methods, along with ablation studies to understand how each factor contributes to the final decision. Storage Decision Accuracy focuses on how correctly the system routes memories into one of its three actions—store for scores greater than or equal to 0.7, compress for scores greater than or equal to 0.4, and discard for scores below 0.4. This is treated as a classification task and evaluated against human-labeled ideal decisions using accuracy and macro-F1 scores. Retrieval Precision measures how effectively stored memories are retrieved from the vector database when needed for generating responses, using standard information retrieval metrics such as precision@k, recall@k, and mean reciprocal rank. Audio-to-Memory Fidelity examines how closely memories extracted from audio match those derived from equivalent text inputs, using semantic similarity and comparative precision-recall analysis. Finally, End-to-End Response Quality evaluates the actual conversational benefit of the system, combining automated relevance metrics with human assessments of personalization, coherence, and usefulness, comparing performance against a baseline system without memory. The framework was further tested on 10 labeled conversations containing 24 ground truth memories, where it achieved 24 true positives, 7 false positives, and no false negatives. This resulted in a precision of 82.4%, recall of 100%, and an F1 score of 89.6%. These results show that the system successfully captured all meaningful memories, achieving perfect recall and ensuring that no critical information was lost—an essential property for systems that rely on continuity and personalization over time. Its ability to handle memory updates was also evident, as repeated or revised user information was correctly integrated by favoring more recent and

relevant versions while compressing outdated details. The main limitation observed was the presence of false positives, where the system occasionally retained temporary emotional expressions or conversational details that do not contribute to long-term understanding. While this slightly reduced precision, it reflects a conservative design choice that prioritizes not losing important information. Moving forward, improvements can focus on more refined filtering strategies that better distinguish between lasting user traits and momentary conversational signals, helping to further improve precision without sacrificing recall. Overall, the framework demonstrates strong performance in capturing, maintaining, and utilizing memory in a way that aligns closely with how meaningful information emerges in natural conversations.

5. CONCLUSION

This work presented the Smart Memory Compression Framework as an approach to handling memory in long conversational AI, with a focus on deciding not just what to remember, but how to manage what is remembered over time. Through the combination of recency, frequency, and relevance, the system demonstrates that memory can be treated as something dynamic rather than static—continuously refined, merged, and updated as conversations evolve. Across varied inputs, including multi-topic text, focused discussions, and natural spoken dialogue, the framework consistently showed its ability to retain meaningful information while reducing unnecessary detail. It proved especially effective at preserving key user attributes and behavioral patterns, while also adapting to repeated or updated information in a way that keeps the memory space both current and coherent. At the same time, the results highlight an important trade-off: in trying to avoid missing important details, the system occasionally retains short-lived conversational signals that do not contribute to long-term understanding.

Overall, this work suggests that effective memory in conversational systems is less about storing more and more about storing better. By continuing to refine how systems distinguish between lasting knowledge and momentary noise, future iterations can move closer to conversations that feel not only context-aware, but genuinely continuous over time.

6. FUTURE ENHANCEMENTS

Looking ahead, there is clear room to make the system feel even more natural and selective in how it handles memory. One direction is to improve its sense of judgment when dealing with short-lived emotions or one-off conversational moments, so it becomes better at letting go of things that do not truly matter in the long run. Another area is making memory more adaptable over time, allowing it to gradually “fade” or evolve as user preferences change, rather than treating everything as equally permanent once stored. There is also potential to make the system more context-aware—understanding not just what is said, but *why* it was said in that moment, which would help it decide whether something reflects a lasting trait or just a temporary situation. Improving how audio is handled is equally important, especially by making the system more tolerant to the natural imperfections of spoken language while still capturing meaning accurately. Finally, future versions can focus on making memory feel more personalized and intuitive, so that it doesn’t just store facts, but builds a deeper sense of the user over time. The goal is to move closer to conversations that feel less like isolated exchanges and more like an ongoing, continuous understanding that grows naturally with every interaction.

7. REFERENCES

[1] A. Vaswani et al., “Attention is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- [2] J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019.
- [3] Y. Liu et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” in *Proc. ACL*, 2019.
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks,” in *Proc. EMNLP*, 2019.
- [5] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *NeurIPS*, 2020.
- [6] V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proc. EMNLP*, 2020.
- [7] T. Brown et al., “Language Models are Few-Shot Learners,” in *NeurIPS*, 2020.
- [8] K. Guu et al., “REALM: Retrieval-Augmented Language Model Pre-Training,” in *Proc. ICML*, 2020.
- [9] M. Roller et al., “Recipes for Building an Open-Domain Chatbot,” in *Proc. EACL*, 2021.
- [10] C. Xiong et al., “Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval,” in *Proc. ICLR*, 2021.
- [11] J. Menick et al., “Teaching Language Models to Support Answers with Verified Quotes,” in *Proc. ICLR*, 2022.
- [12] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models,” in *Proc. ICLR*, 2023.
- [13] S. Shuster et al., “Dialogue Retrieval with Context Dependent Memory,” in *Proc. ACL*, 2021.
- [14] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” in *Proc. ICML*, 2021.
- [15] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *NeurIPS*, 2022.
- [16] S. Sanh et al., “Multitask Prompted Training Enables Zero-Shot Task Generalization,” in *Proc. ICLR*, 2022.
- [17] T. Schick et al., “Self-Consistency Improves Chain of Thought Reasoning,” in *Proc. ACL*, 2023.
- [18] H. Chung et al., “Scaling Instruction-Finetuned Language Models,” in *Proc. ACL*, 2022.
- [19] J. Dodge et al., “Measuring the Carbon Intensity of AI in Cloud Instances,” in *Proc. FAccT*, 2022.
- [20] Y. Tay et al., “Long Range Arena: A Benchmark for Efficient Transformers,” in *Proc. ICLR*, 2021.
- [21] O. Press et al., “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” in *Proc. ICLR*, 2022.
- [22] J. Lample et al., “Cross-Lingual Language Model Pretraining,” in *NeurIPS*, 2019.
- [23] D. Bahdanau et al., “Neural Machine Translation by Jointly Learning to Align and Translate,” in *Proc. ICLR*, 2017 (updated version in circulation).
- [24] A. Radford et al., “Whisper: Robust Speech Recognition via Weak Supervision,” in *Proc. ICML Workshops*, 2023.
- [25] J. Gao et al., “Neural Approaches to Conversational AI,” in *Proc. SIGIR*, 2021.

[26] Z. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models,” in *Proc. ICLR*, 2021.

[27] M. Lewis et al., “BART: Denoising Sequence-to-Sequence Pre-training,” in *Proc. ACL*, 2020.

[28] J. Ni et al., “Learning to Retrieve, Read, and Answer: End-to-End QA Systems,” in *Proc. ACL*, 2021.

[29] S. Ramesh et al., “Hierarchical Memory Networks for Document-Level Understanding,” in *Proc. EMNLP*, 2022.

[30] D. Guo et al., “Long-Term Memory for Conversational AI Systems,” in *Proc. ACL Workshops*, 2023.